CUCEI
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Materia:

Ingeniería de software

1er Milestone.

23/02/2026

Actividad:

Análisis de Requerimientos v1.0

Profesor:

Ruben Adrian Gil Rivera

Integrantes:

Ricardo Nieto Campos
Ximena Montserrat González Sandoval
David Isaac Martinez Moreno
Brenda Sarai Ramirez Aguilar
Julia Vanessa Cortes Quezada

Índice

Introducción 3
Alcance del proyecto 3
Descripción general 4
Objetivos **5**
Objetivo General 5
Objetivos Específicos 5
Viabilidad **6**
Viabilidad Técnica 6
Viabilidad Operativa 6
Viabilidad Económica 6
Requerimientos Funcionales (RF) 7
Requerimientos No Funcionales (RNF) 9
Entorno y Herramientas de Desarrollo 10
Riesgos 12
Diagrama de Gantt y planeación de actividades del proyecto (timing plan) 14
Análisis y Arquitectura 14
Diseño por funcionalidad 14
Desarrollo iterativo/ Sprints (Marzo-Abril) 14
Pruebas (QA) y pulido (Abril - Mayo) 14
Despliegue y Cierre(Mayo) 14

Introducción

El propósito de este documento es definir de manera clara y estructurada los requerimientos para el desarrollo de la plataforma llamada **Smart Learn**. Este sistema surge como una solución tecnológica ante la dificultad de los estudiantes para organizar recursos de aprendizaje dispersos y la falta de métodos efectivos para la retención de información a largo plazo a base de herramientas de autoaprendizaje.

El proyecto se centra en la creación de un entorno centralizado que combine la gestión de contenidos multimedia con el método científico e información. Se busca un equilibrio entre una arquitectura de software robusta y una implementación que sea escalable a largo plazo, permitiendo agregar funcionalidades que se requieran con el tiempo conforme a opiniones o alcances del sistema.

Alcance del proyecto

El sistema será una aplicación web que permitirá a estudiantes Universitarios a dar gestión y organización a sus recursos de estudio (archivos, enlaces, documentos, libros) y usar técnicas de **Repetición Espaciada** o “Flashcards” para optimizar el aprendizaje y las sesiones de estudio entre materia y materia.

- **Incluye:** Login de Usuarios, Gestión de mazos, algoritmo de repaso SM-2 (para calcular los intervalos de los repasos), programación de sesiones de repaso según la dificultad y visualización del progreso sobre el material de estudio.
- **No incluye:** App móvil, sistema de pago, integración a Redes Sociales, suscripciones o planes de estudio premium basados en sistemas de pago.

Descripción general

El sistema de **Smart Learn** se implementa como una solución integral de gestión de conocimiento diseñada específicamente para el aprendizaje autodidacta en la era digital.

Esto a base del almacenamiento de la información y los enlaces de interés de manera organizada y con la característica de poder implementar notas de estudio para una más sencilla curva de aprendizaje permitiendo que el usuario no solo acumule recursos, sino que los asimile mediante un flujo de trabajo estructurado y respaldado por principios de la psicología cognitiva.

El núcleo funcional de la aplicación se basa en el concepto de **Micro-Learning**. Esta metodología fragmenta temas complejos en pequeñas cápsulas de información como un video específico de YouTube, un artículo técnico o un manual en PDF que resultan más fáciles de procesar. El sistema permite al estudiante organizar estas unidades en módulos temáticos, creando una jerarquía lógica que facilita la navegación y el enfoque mental. Al centralizar fuentes de diversas procedencias en una sola interfaz, se reduce significativamente la carga cognitiva y la fatiga que produce el cambio constante entre pestañas y aplicaciones.

Aunque el sistema maneja flujos de datos complejos entre usuarios y recursos. El objetivo es que la tecnología sea transparente para el usuario, permitiendo que la atención se centre en el contenido y no en el manejo de la herramienta.

Finalmente, el desarrollo se apoya en tecnologías de vanguardia que garantizan la escalabilidad y la portabilidad. Al ser una aplicación web, Smart Learn elimina las barreras de instalación, permitiendo el acceso desde cualquier navegador moderno. La combinación de una base de datos persistente y un backend ágil que asegura que un estudiante pueda guardar links url de un video en su computadora junto con otros recursos importantes y repasar las tarjetas con preguntas asociadas al tema de interes mientras se desplaza.

Objetivos

Objetivo General

Desarrollar una plataforma web de **Micro-Learning** que permita a los estudiantes centralizar recursos académicos y optimizar el seguimiento de su progreso mediante herramientas de organización.

Objetivos Específicos

- **Gestión de Datos:** Implementar un sistema de autenticación y persistencia que permita a cada usuario administrar sus propias materias y materiales de forma aislada y segura.
- **Centralización Documental:** Desarrollar un módulo para la carga y visualización de archivos PDF y enlaces externos, permitiendo una clasificación manual por categorías temáticas.
- **Control de Avance:** Diseñar una lógica de estados (Pendiente, En Progreso, Completado) para calcular y visualizar el porcentaje de progreso por cada materia creada.
- **Refuerzo Académico:** Proveer herramientas de autoevaluación mediante la creación manual de flashcards y cuestionarios tipo test.

Viabilidad

Viabilidad Técnica

El proyecto es **altamente viable** debido a la simplificación del alcance original al modelo MVP:

- **Tecnologías:** El uso del stack **PERN (PostgreSQL, Express, React, Node.js)** facilita la comunicación cliente-servidor y el manejo de datos relacionales necesarios para los diagramas UML que están construyendo.
- **Infraestructura:** La manipulación de archivos se restringe a formato PDF para simplificar la extracción de texto y el almacenamiento.

Viabilidad Operativa

La plataforma responde a una necesidad real del equipo y de otros estudiantes:

- **Usabilidad:** El diseño propuesto por Vanessa y Brenda asegura que cualquier estudiante pueda usar la herramienta sin capacitación previa.
- **Mantenibilidad:** Al seguir una arquitectura limpia, el código será fácil de depurar por David y Ricardo durante el semestre.
- **Impacto:** El sistema de "Rachas" y "Repetición Espaciada" (si se llega a implementar como mejora) tiene una base pedagógica que asegura que la herramienta sea útil más allá de la entrega académica.

Viabilidad Económica

El costo financiero del proyecto es **cero (0)**:

- **Herramientas Open Source:** Se utilizarán licencias gratuitas de lenguajes y frameworks.
- **Cloud Hosting:** El despliegue se realizará en plataformas con capas gratuitas (como Vercel para el front, Render para el back y Supabase para la base de datos).
- **Consumo de APIs:** Se aprovecharán los créditos de prueba o capas gratuitas de APIs de IA para las demostraciones en clase.

Requerimientos Funcionales (RF)

ID	Requerimiento	Descripción	Prioridad
RF-01	Registro de Usuarios	Login con correo electrónico y contraseña	ALTA
RF-02	Inicio de Sesión	El sistema debe validar las credenciales del usuario para el acceso	ALTA
RF-03	Crear Mazo	Crear categorías (Mazos) para organizar el material	ALTA
RF-04	Agregar Recurso	Poder guardar URLs, PDFs asociados a un Mazo	MEDIA
RF-05	CRUD	El usuario podrá editar o eliminar recursos previamente guardados	MEDIA
RF-06	Crear FlashCard	El usuario podrá hacer tarjetas(R) dentro de un Mazo	ALTA
RF-07	Modo Repaso	Mostrará las tarjetas ocultando las respuestas hasta que el usuario interactúe	ALTA
		El usuario podrá calificar la	

RF-08	*Calificación*	*dificultad (Fácil, Medio, Difícil) tras ver la respuesta*	*ALTA*
RF-09	*Visualización de Progreso*	*El sistema mostrará gráficas de barras con la actividad de los últimos días*	*ALTA*
RF-10	*Límite de memoria*	*Establecer un límite de 10MB para optimizar el almacenamiento de los recursos*	*ALTA*

Requerimientos No Funcionales (RNF)

ID	Requerimiento	Descripción	Prioridad
RFN-01	Usabilidad y Diseño (UX/UI)	La interfaz deberá ser limpia y minimalista, priorizando la facilidad de navegación.	Alta
RNF-02	Disponibilidad y Despliegue	La aplicación será desplegada en servicios de nube de acceso gratuito. El objetivo es que la plataforma sea accesible vía URL las 24 horas del día para su evaluación y uso real.	Alta
RNF-03	Persistencia de Datos	Se empleará una base de datos que garantice la integridad de la información. El progreso del usuario no debe perderse en ninguna circunstancia tras el cierre de sesión.	Alta
RNF-04	Seguridad Básica	Las credenciales de acceso deben ser protegidas.	Alta

Entorno y Herramientas de Desarrollo

Para garantizar la escalabilidad, mantenibilidad y colaboración eficiente del equipo, elegimos varias tecnologías basadas en Stack PERN y Herramientas Ágiles, de igual manera, si en el futuro nos vemos en la necesidad de cambiar, agregar o prescindir de alguna de estas tecnologías, iremos actualizando nuestro Stack de tecnologías a usar.

- **Stack de Tecnologías PERN**

Capa	*Tecnología*	*Justificación*
Frontend	*React.js, HTML, CSS*	React nos permite la ceración de interfaces dinámicas basadas en componentes reutilizables (para nuestros flashcards), mientras que para el resto del esqueleto de la plataforma por medio de HTML y CSS nos centramos en el diseño visual para la visualización final del cliente
Estilos (UI)	*Tailwind CSS*	Es un framework de utilidad para diseñar la interfaz rápidamente sin salir de HTML, para tener consistencia visual y diseño responsive
Backend	*Node.js + Express*	Es un entorno de ejecución ligero y orientado a eventos. Y Express facilita la creación de Endpoints RESTfulpara comunicar el cliente con la base de datos y la API de IA
Base de Datos	*PostgreSQL*	Es un motor de base de datos relacional robusto. Lo vamos a usar para mantener la integridad de las relaciones entre los Usuarios, Mazos y Tarjetas
Nube/BaaS	*Supabase*	Es una plataforma que provee PosgreSQL como servicio, además de gestionar la autenticación y el almacenamiento de archivos (Buckets para los PDFs)

Herramientas de Gestión y Colaboración.

- **Versionamiento: Git y GitHub**. Se utiliza un flujo de trabajo basado en ramas y de forma paralela (Feature Branch Workflow) para evitar conflictos en el código principal (el main del repositorio)
- **Gestión de Proyecto: Notion.** Utilizando para el seguimiento de tareas, documentación de requerimientos y el diagrama de Gantt
- **Diseño: Figma.** Herramienta para el prototipado de alta fidelidad y diseño de la experiencia del usuario UX/UI antes de la codificación
- **Comunicación API: Postman.** Para probar los endpoints del Backend de manera aislada antes de integrarlos con el Frontend

Riesgos

ID	ÁREA/ CLASIFICACIÓN	RIESGO	MEDIDA DE CONTENCIÓN	LIKELIHOOD	SCORE
1.	Project management	Desviación del alcance (Scope Creep) al intentar añadir multimedia compleja.	Apegarse estrictamente al modelo MVP definido en este documento	Likely (4)	12 (High)
2.	Usuario	Mal comportamiento del usuario (Mal uso del sistema o carga de archivos no aptos)	Analizar diagramas de casos de uso y especificaciones de software	Unlikely (2)	6 (Moderate)
3.	Project Management	Retraso en el diseño de la Arquitectura de Software.	Priorizar el modelado de la base de datos para no detener el desarrollo del Backend.	Likely (4)	12 (High)
4.	Riesgo técnico	Fallos en el cálculo del Algoritmo SM-2 para los intervalos de repaso.	Realizar pruebas unitarias exhaustivas sobre la lógica matemática del algoritmo.	Possible (3)	9 (Moderate)
5.	Infraestructura	Pérdida de recursos persistentes (PDFs o datos de progreso).	Implementar políticas de respaldo y usar bases de datos con alta disponibilidad.	Very Unlikely (1)	4 (Low)
6.	Riesgo técnico	Incompatibilidad de lectura en archivos PDF con formato de imagen.	Validar la estructura del PDF antes del procesamiento o incluir una alerta de error al usuario.	Possible (3)	9 (Moderate)
7.	Riesgo técnico	Alucinaciones de la IA (Generación de preguntas con respuestas incorrectas o sin sentido).	Implementar un sistema de validación humana donde el usuario pueda editar o reportar errores en el quiz.	Possible (3)	9 (Moderado)
8.	Seguridad	Inyección de Código en Metadatos (Carga de archivos maliciosos ocultos en PDFs).	Sanitizar todos los archivos en el servidor y utilizar almacenamiento aislado como Supabase o AWS	Very Unlikely (1)	4 (Low)

			S3.		
9.	Riesgo técnico	Caída o interrupción de los servicios de terceros en la nube (Supabase) que impida el acceso al sistema o a la base de datos	Implementar manejo de errores en el Frontend (pantalla de "Servicio no disponible").	Possible (3)	9 (Moderate)

Diagrama de Gantt y planeación de actividades del proyecto (timing plan)

El cronograma del proyecto ha sido diseñado utilizando un enfoque iterativo e incremental, dividiendo el ciclo de vida del software en fases secuenciales y sprints paralelos. Esta estructura permite al equipo minimizar riesgos, asegurar entregas tempranas del valor y mantener una coordinación eficiente entre los integrantes del equipo de Frontend y Backend.

Análisis y Arquitectura

Como objetivo se busca la definición de requerimientos (SRS), diseño del modelo relacional (Base de datos), y diagramas de arquitectura. En esta etapa inicial, el esfuerzo se concentra en la célula de datos y backend para establecer cimientos sólidos antes de escribir código.

Diseño por funcionalidad

El objetivo del diseño por funcionalidad es la creación de wireframes, prototipos de alta fidelidad (UI/UX) y diseño de la experiencia del usuario. Mientras el equipo creativo (Diseño, Frontend), define las interfaces, el equipo técnico configura los entornos de desarrollo(Setup y configuration management en Github).

Desarrollo iterativo/ Sprints (Marzo-Abril)

Se dividió la codificación en sprints para aplicar paralelismo. Mientras backend construye la lógica, Frontend maqueta las vistas del programa.

- Sprint 1 (Esqueleto y CRUD): enfocado en el módulo base (Registro, Login, subida de PDFs y creación de Mazos).
- Sprint 2 (Core y Algoritmos): la integración de la API de inteligencia artificial para generación de preguntas y programación del algoritmo de repetición espaciada(SM-2).

Pruebas (QA) y pulido (Abril - Mayo)

Su objetivo es la ejecución de casos de prueba, integración del Dashboard de estadísticas y corrección de bugs(Debug). Aquí el área de QA toma el liderazgo para estresar el sistema, mientras los desarrolladores aplican parches de solución.

Despliegue y Cierre(Mayo)

Para esta parte se busca subir el proyecto a un entorno de producción (Nube), congelar el código (CodeFreeze) y generar los manuales finales para la presentación.

Para optimizar los tiempos de entrega, el cronograma emplea paralelismo de tareas. Por ejemplo, la maquetación del Frontend y la creación de la API ocurren de manera simultánea gracias a la previa definición de contratos (Endpoints). Además, se han contemplado holguras de tiempo (buffers) durante el cruce con el periodo vacacional (Semana Santa) y antes de la entrega final para absorber posibles retrasos técnicos, como la integración con la API de IA.

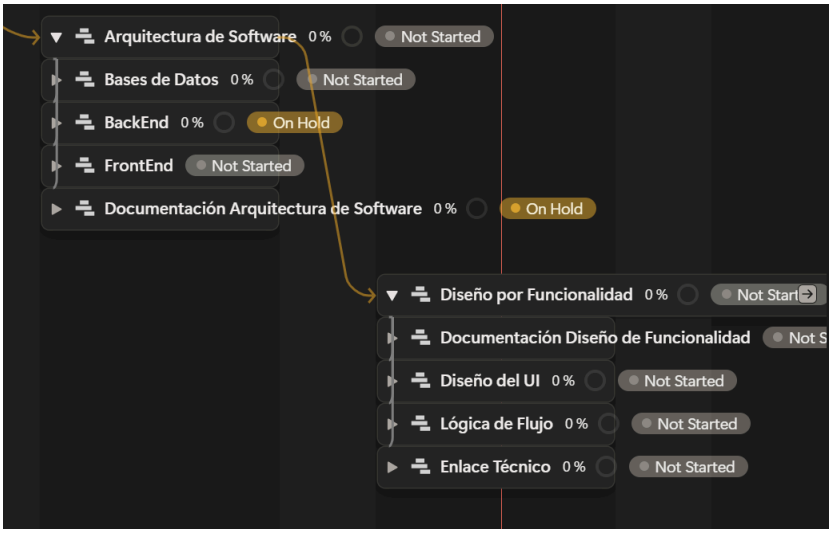

Figura 0. *Uso de paralelismos en la división de tareas en la Arquitectura de Software y el Diseño por Funcionalidad.*

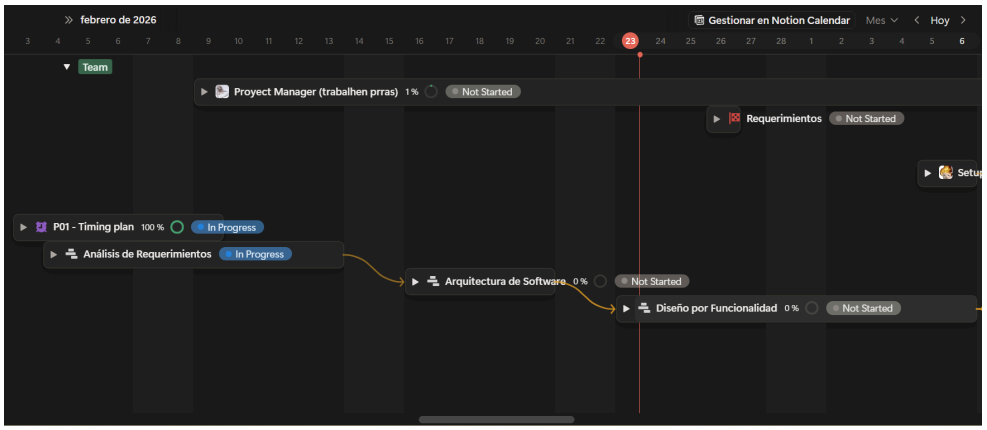

Figura 1. *Diagrama de Gantt del proyecto, ilustrando la ruta crítica y la asignación de Sprints.*

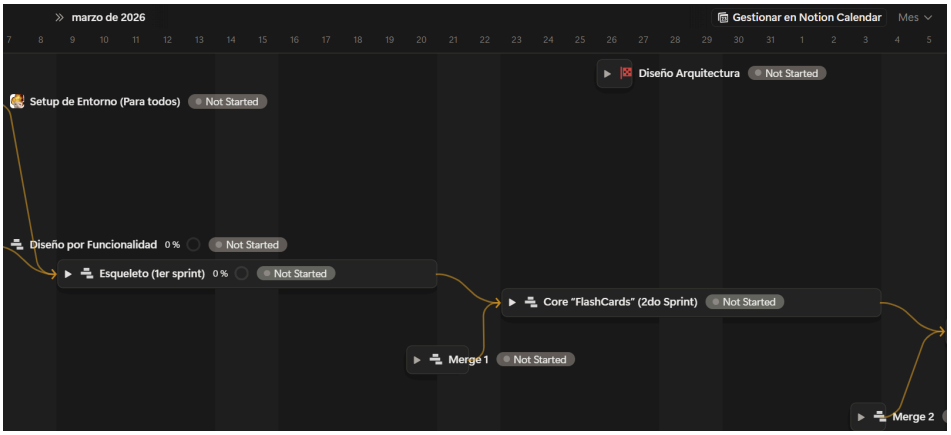

Figura 2. *Diagrama de Gantt del proyecto, se muestra el primer Sprint y el segundo Sprint, con una tarea de Merge que es la etapa en la que se mezclan las ramas de código independientes elaboradas en paralelo, en una rama principal para comenzar el segundo Sprint.*

Figura 3. *Se muestra el Dashboard y pulido, con el tercer Merge para comenzar con el diseño de pruebas, al terminar se ejecuta la entrega de los diseños para continuar con la ejecución.*

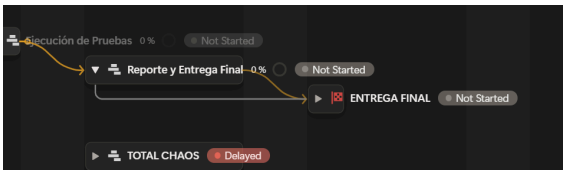

Figura 4. *Visualización de las últimas tareas, el reporte final y la entrega final del proyecto.*

Figura 5. *Cada tarea se diseñó con una fecha de inicio y finalización, también se asignó a cada compañero y se agregó un status para que todos los integrantes puedan ver el avance de la tarea.*